File Transfer System Based on Simple Transfer and Exchange Protocol (STEP)

Jinlong.Huang
2364343
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
Jinlong.Huang23@student.xjtlu.edu.cn

Leyi.Li
2362141
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
Leyi.Li23@student.xjtlu.edu.cn

Nuoqian.Xu
2363177
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
Nuoqian.Xu23@student.xjtlu.edu.cn

Shuheng.Hu
2362132
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
Shuheng.Hu23@student.xjtlu.edu.cn

Zeyu.Yin
2360955
School of Advanced Technology
Xi'an Jiaotong - Liverpool University
Suzhou, China
Zeyu.Yin23@student.xjtlu.edu.cn

***Abstract*—This project presents a robust file transfer system using the STEP protocol within a client-server framework. Key features include user authentication, multi-threaded chunked uploads, and automated integrity verification via MD5 checksum. Compared to FTP-based approaches, adaptive retransmission and token-based authentication improve reliability and security. Experimental evaluation demonstrates enhanced efficiency and stability under localhost conditions. Performance bottlenecks and future optimizations are also discussed.**

***Index Terms*—C/S, Socket, TCP, STEP, Multithreading**

I. Introduction

A. Background

The File Transfer Protocol (FTP), introduced by Abhay Bhushan in 1971 for ARPANET [1], has since been enhanced with features such as passive mode, security extensions, and IPv6 support [2]. Despite its age, FTP remains a core protocol for reliable file transfer. Inspired by FTP and within the CAN201 course, our group designed the Simple Transfer and Exchange Protocol (STEP), a TCP/IP application-layer protocol for fast, secure, and reliable network file transmission.

B. Task Specification

STEP is a stateless TCP-based protocol that uses JSON format to enable data and file exchange between clients and servers.In our project, server accepts any number of authorized clients to upload files. Additionally,Additionally, we designed a multi-threaded client that parallelizes block uploads to improve transfer efficiency.

C. Practice Relevance

This project provides an effective platform for online file management and content distribution for small businesses and individuals, delivering secure and efficient file transfer and management. Features such as user authentication, storage, and download support improve transfer efficiency while protecting data security and integrity.

D. Challenge and Contributions

During implementation and testing, we identified challenges related to increasing transfer speed as a bottleneck. To accommodate the varied schedules of our team members, we used comprehensive documentation and applied agile development principles throughout project development lifecycle. Finally, our project proposed a protocol that can serve small businesses and individuals by offering secure, efficient file management and distribution services.

II. Related Work

Reliable and secure file transfer has received significant attention, with recent work emphasizing chunk-based transmission, token-based authentication, and secure deduplicated storage. Variable-size chunking adapts to file content and improves performance [3], while JWT-based schemes provide stateless authentication suitable for distributed systems [4]. In terms of storage efficiency, deduplication substantially reduces space, bandwidth, and operational costs [5]. Our project integrates these three techniques to achieve reliable and efficient file transfer.

III. Design

A. Server Debugging

Server debugging follows below progressive methodology to assure server output as expected:

- **Unit Testing**: Each functional component such as authentication, block parsing, and file writing was individually validated through unit testing with controlled inputs.
- **Protocol Layer Verification**: Message handling and protocol framing were instrumented with verifications to ensure the correctness of JSON header extraction, binary payload positioning, and length interpretation over TCP streams.
- **Robustness Testing**: To evaluate performance under operational stress, we simulated multi-client concurrent

upload situations with deliberate network impairments, specially packet loss and out-of-order delivery.

B. Network Architecture Design

This project adopts a TCP-based client–server architecture using the STEP protocol for file upload and download. As shown in Figure 1, the server listens on port 1379 and manages multiple concurrent client connections for simultaneous transfers.

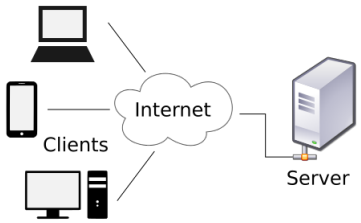

Fig. 1. Client-Server Application

C. Client Sequence Diagram

1) ***Authentication***: Users can choose to either run the Python program for an interactive login or input commands directly in the terminal for a non-interactive login. If the user selects the interactive mode, they are required to log in or create an account by providing the parameters `username` and `password`. The user information will be saved in `user.json`.

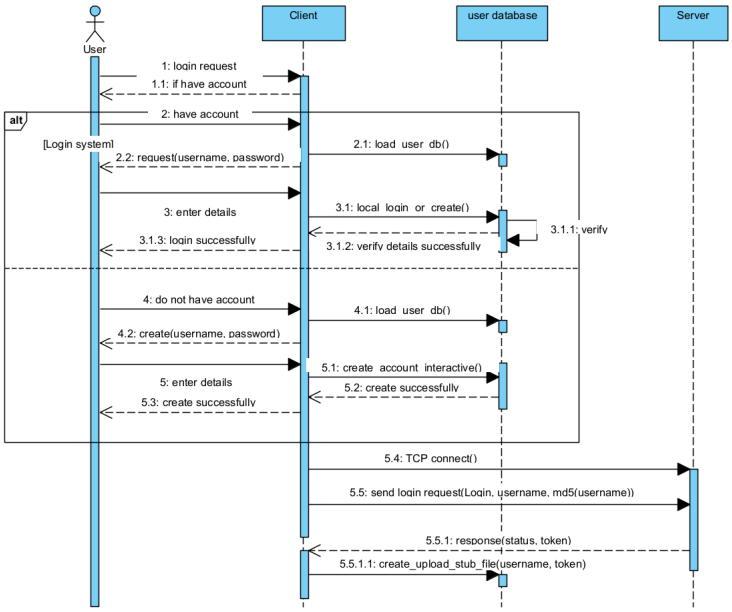

Fig. 2. Interactive Login System

To establish a connection with the server, the client sends `MD5(username)` request to the server and receives an encoded token containing the username and a hash checksum string if authentication is successful (status==200). The overall login process is illustrated in Fig. 2.

2) ***File Upload***: This module uploads files to the server while ensuring data integrity. The user provides `user_input` through `ask_upload_path`, then the client sends a STEP message with `OP_SAVE`, `TYPE_FILE`, `file_size`, and `file_key` to obtain the upload plan. After verifying the file and reading the plan, the client starts the multi-threaded upload. When all chunks are uploaded, the server returns the file's MD5, which is compared with the local MD5 to determine and print the final upload result.

D. Program Flow Chart

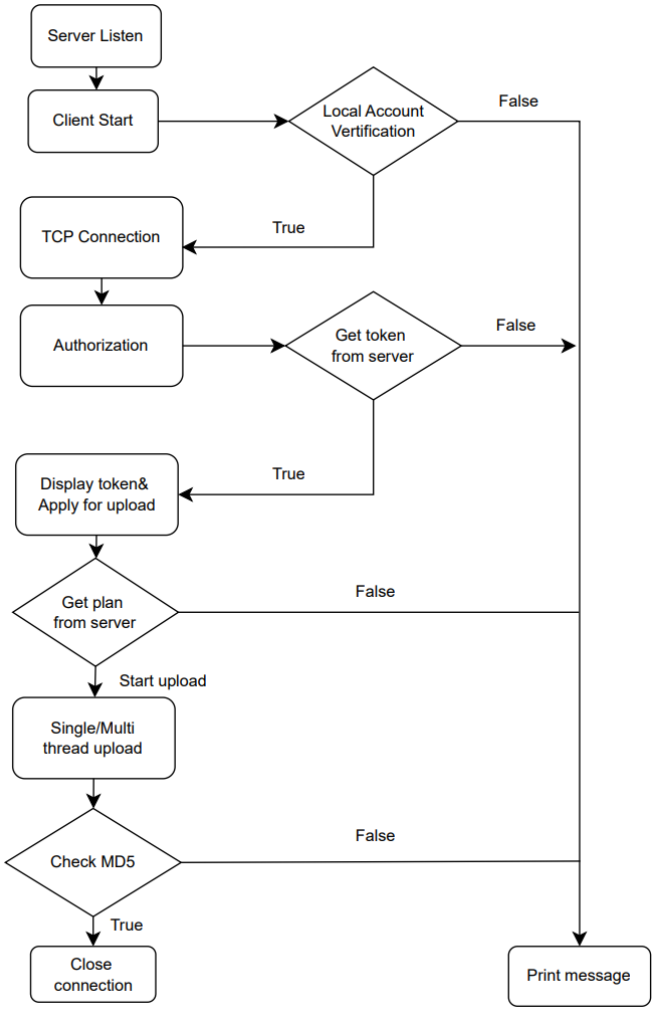

Fig. 3. Program Flow Chart

E. Algorithm

Figures 4 and 5 illustrate the pseudo-code of the client-side processes. Figure 4 shows the authorization algorithm, which handles user login and account creation. Figure 5 presents the file upload algorithm, detailing the process of uploading files to the server, including block-wise transfer and MD5 integrity verification.

Algorithm 1: Client-Side Authorization Algorithm

```
Server IP, Server Port, Output File Path Valid Token
(if success) or Null (if failure)
Initialize local user database (users.json);
if No local accounts exist then
    Prompt user to create new account
     (username/password);
    Validate username (no whitespace, length  64);
    Validate password (non-empty, matching
     confirmation);
    Save account to local database;
end
else
    Prompt user to select "login" or "create account";
    if User selects "create account" then
        Execute account creation process (same as
         above);
    end
    else if User selects "login" then
        Prompt for username and password;
        if Username not in local database then
            Return failure (username not found);
        end
        if Password does not match local record then
            Return failure (incorrect password);
        end
    end
end
Generate validate_token = MD5(username);
Create login request JSON: {type: AUTH, operation:
 LOGIN, direction: REQUEST, username, password:
 validate_token};
Establish TCP connection to server (IP:Port);
Send login request packet (per STEP protocol);
Receive server response;
if Response status = 200 then
    Extract token from response;
    Save login info (username, token, server IP:Port) to
     output file;
    Return token;
end
else
    Return failure (server authentication failed);
end
TCP connection;
```

Fig. 4. Authorization Algorithm

Algorithm 2: File Upload Algorithm

```
Valid Token, Username, File Path, Server IP, Server
Port Upload success status (Boolean)
Re-establish TCP connection to server (IP:Port);
if File does not exist or is invalid then
    Return failure (file not found);
end
Calculate file size;
Create save request JSON: {type: FILE, operation:
 SAVE, direction: REQUEST, token, username, size:
 file_size, key: filename};
Send save request packet;
Receive upload plan response;
if Response status  200 or missing required fields (key,
 block_size, total_block) then
    Return failure (invalid upload plan);
end
Extract upload plan: {key, block_size, total_block};
Calculate local file MD5;
Initialize progress bar (total = total_block);
Open file in binary read mode;
for block_index = 0 to total_block - 1 do
    Read block_data (size = block_size) from file;
    Create upload request JSON: {type: FILE,
     operation: UPLOAD, direction: REQUEST, token,
     username, key, block_index};
    Send upload packet (JSON + block_data);
    Receive block upload response;
    if Response status  200 then
        Return failure (block upload failed);
    end
    Update progress bar;
end
file;
if Server returns MD5 then
    if Server MD5 == Local MD5 then
        Return success (upload completed);
    end
    else
        Return failure (MD5 check failed);
    end
end
else
    Return failure (server MD5 missing);
end
TCP connection;
```

Fig. 5. File Upload Algorithm

IV. Implementation

A. Development Environment

During the project, several team members contributed to coding. The development was primarily carried out on the computer listed in Table I.

- **Used Python Modules**: argparse, hashlib, json, os, struct, threading, time, socket

B. Programming Skills

- **Modular Design**: The code is managed in a modular manner by reusable functions with low coupling and high cohesion.
- **Concurrency**: The transfer system has ability to handle multi-thread procedures from different sessions while keeping security and efficiency at same time.

TABLE I
DEVELOPMENT ENVIRONMENT

	Specification
OS	macOS Sequoia 15.5
CPU	Apple M4 Pro
RAM	48 GB
IDE	Pycharm 2024.1.1
Python	Python 3.8.20

C. Steps of Implementation

1) ***STEP Protocol Analysis***: Implementing a client for the STEP protocol necessitates a comprehensive analysis of its specifications. STEP messages leverage JSON for data exchange and binary data for file transfers. This structure ensures clear delineation between control information (JSON) and bulk data (binary) within a single TCP stream.

2) ***Development Stage***:

Initial efforts focused on fixing bugs within the existing server-side code, as showcased in `server.py`. This involved addressing missing import statement, misspelled function name and unsuitable parameters.

Following successfully the server-side code and a comprehensive understanding of the STEP protocol, we focused on client-server communication functions, as shown in `client.py`. The iterative process of writing and testing these functions facilitated a deeper comprehension of the STEP protocol's mechanics and the server's operational behavior. Subsequently, the codes were refactored using abstraction principles to achieve a modular design and optimize underlying algorithms.

D. Program Implementation

1) ***Authorization Function***: This function handles user authentication by logging into the STEP server and returning an authorization token for subsequent operations. It constructs a LOGIN message in JSON format, requiring "username" and "password" fields. The password is an MD5 hash of the username. Additionally, the "type" field is set to "AUTH" and the "operation" field to "LOGIN" to identify the message's purpose. This complete LOGIN message is then transmitted to the server via a TCP socket. Upon successful authentication, the server returns the corresponding user token.

2) ***File Uploading Function***: To upload a file, the client sets `FIELD_KEY` as the file name and sends an `OP_SAVE` request to obtain an upload plan. The server replies with `FIELD_TOTAL_BLOCK` and `FIELD_BLOCK_SIZE`, which define the number and size of chunks. The client then uploads each chunk sequentially, and the server verifies the frame structure and acknowledges each block before the next is sent. After the final block, the client computes the file's MD5 hash and compares it with the server's checksum to verify transmission integrity.

E. Difficulties and Solutions

1) ***Teamwork***: In our teamwork, many problems occur. Since our team members have different time schedule, we arranged weekly meetings to discuss ideas and align progress with each other.

2) ***Performance Bottleneck***: During the development of the STEP-based file transfer system, we found that multi-threaded uploads often underperformed, especially as thread count increased. Small files incurred thread and connection overhead, while large files faced network and TCP congestion. Thus, the system defaults to single-threaded uploads, using four threads only for files larger than 1 GB to ensure stable, sequential TCP utilization and predictable throughput.

V. TESTING AND RESULT

A. Testing Environment & Metrics

The experiments were conducted inside a virtualized environment on a Windows host machine (see Table II), where both the client and server ran within the same virtual machine. This setup minimizes external network effects, ensuring that observed differences reflect only the implementation.

TABLE II
TESTING ENVIRONMENT (HOST WINDOWS PC)

Component	Specification
CPU	13th Gen Intel Core i9-13900HX
RAM	16 GB
OS	Windows 11 Home
Python	3.10.12
Network	Wi-Fi 6

To evaluate upload performance, the client recorded the execution time of each upload task, along with the resulting throughput and average block latency. We tested both single-threaded and four-thread uploads using files of 5 MB, 10 MB, 50 MB, 100 MB, and 500 MB. Each configuration was executed three times, and the average results were reported.

B. Localhost Testing

We first executed the server on the local virtual machine to verify that the program functions correctly. As illustrated in Fig. 6, upon running `python3 server.py`, the server successfully displays the message "Server is ready!"

```
g@g-virtual-machine:~/can201$ python3 server.py
2025-11-14 17:48:50-STEP[INFO] Server is ready! @ server.py[663]
2025-11-14 17:48:50-STEP[INFO] Start the TCP service, listing 1379 on IP All ava
ilable @ server.py[664]
```

Fig. 6. Running server.py

In the second step, we ran `python3 client.py --server_ip <IP> --id <USER_ID> --f <FILE_PATH>` in the terminal. Upon execution, the client output `Token:{token}` to indicate successful authentication, as shown in Fig. 7.

After a successful login, the terminal displayed the file upload progress as well as the final upload result, as shown in Fig. 8.

For evaluation purposes, we can append the `--measure` flag to the client command, which enables the calculation of total upload time, throughput, and average block latency. An example of the measurement output is shown in Fig. 9.

```
g@g-virtual-machine:~/can201$ python3 client.py --server_ip 127.0.0.1 --port 137
9 --id 2362141 --f /home/g/can201/test_10MB.bin
Connect to server 127.0.0.1
Requesting to log in...
Successfully get response from server:{'token': 'MjM2MjE0MS4yMDI1MTExNDE5MzMzOC5
sb2dpbi5kNzlkYTM0Yzk2OTgzZjNhYTk0YjEwMjJiNTdiYThjNQ==', 'operation': 'LOGIN', 'd
irection': 'RESPONSE', 'status': 200, 'status_msg': 'Login successfully', 'type'
: 'AUTH'}
Successfully login!
Token: MjM2MjE0MS4yMDI1MTExNDE5MzMzOC5sb2dpbi5kNzlkYTM0Yzk2OTgzZjNhYTk0YjEwMjJiN
TdiYThjNQ==
```

Fig. 7. Login Successful

```
Reconnected to server for upload
Requesting upload plan for file: test_10MB.bin (size: 10485760 bytes)
Upload plan obtained:
Key: test_10MB.bin
Block size: 20480 bytes
Total blocks: 512
Uploading test_10MB.bin (key: test_10MB.bin):   0%|                          | 0/512
Uploading test_10MB.bin (key: test_10MB.bin):   8%|                | 42/512 [00:00<0
Uploading test_10MB.bin (key: test_10MB.bin):  16%|                | 83/512 [00:00<0
Uploading test_10MB.bin (key: test_10MB.bin):  24%|               | 123/512 [00:00<0
Uploading test_10MB.bin (key: test_10MB.bin):  32%|               | 163/512 [00:00<0
Uploading test_10MB.bin (key: test_10MB.bin):  39%|               | 202/512 [00:00<0
Uploading test_10MB.bin (key: test_10MB.bin):  47%|               | 241/512 [00:00<0
Uploading test_10MB.bin (key: test_10MB.bin):  54%|               | 279/512 [00:00<0
Uploading test_10MB.bin (key: test_10MB.bin):  62%|               | 315/512 [00:00<0
Uploading test_10MB.bin (key: test_10MB.bin):  69%|               | 351/512 [00:00<0
Uploading test_10MB.bin (key: test_10MB.bin):  76%|               | 388/512 [00:01<0
Uploading test_10MB.bin (key: test_10MB.bin):  83%|               | 427/512 [00:01<0
Uploading test_10MB.bin (key: test_10MB.bin):  91%|               | 464/512 [00:01<0
Uploading test_10MB.bin (key: test_10MB.bin):  98%|               | 500/512 [00:01<0
0:00, 351.55block/s]
 File upload completed successfully! MD5 check passed.
   Local MD5: f1c9645dbc14efddc7d8a322685f26eb
   Server MD5: f1c9645dbc14efddc7d8a322685f26eb
Uploading test_10MB.bin (key: test_10MB.bin): 100%|               | 512/512 [00:01<0
0:00, 338.27block/s]
File upload task finished!
Disconnected from server
```

Fig. 8. Uploading a 10MB file

C. *Testing Result*

The code defaults to single-threaded file uploads, while multi-threaded uploads can be enabled by modifying the configuration to spawn multiple threads, each using its own TCP connection. Experimental results indicate that single-threaded uploads generally outperform four-threaded uploads across files of different sizes (see Fig. 10).

For small files (5–10 MB), the overhead of thread management and multiple TCP connections reduces throughput and increases block latency.

For medium files (50 MB), the relative impact of this overhead diminishes, and completion times for single- and multi-threaded uploads become comparable.

For large files (100–500 MB), multi-threaded uploads may again degrade in performance. With multiple TCP connections, network contention, kernel buffer pressure, and TCP congestion control can limit throughput.

In contrast, single-threaded uploads over a single TCP connection maintain stable, sequential socket utilization, consistently achieving higher efficiency. Overall, in this architecture, multi-threading does not always provide a performance advantage, and network serialization and TCP flow control dominate throughput, making single-threaded transfers more predictable and effective.

VI. Conclusion

This study demonstrates the feasibility of implementing a secure, efficient, and reliable file transfer system based on the STEP protocol. The proposed framework integrates user authentication, single-threaded and multi-threaded uploading,

```
=== Upload stats ===
Total time: 1.434 s
Throughput: 6.974 MB/s
Avg block latency: 0.0027 s  Median: 0.0025 s  Min: 0.0009 s  Max: 0.1037 s
```

Fig. 9. Performance of the 10MB file upload

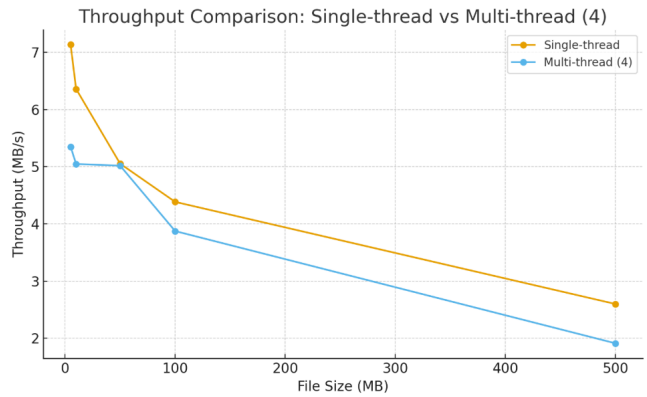

Fig. 10. Single-thread vs Four-thread

and integrity verification, ensuring stable data delivery across files of varying sizes. Experimental validation confirms that single-threaded uploads consistently achieve higher efficiency, while multi-threaded uploads introduce overhead that can reduce performance, particularly for small and large files. Future work will focus on optimizing concurrency control, improving thread management, and integrating encryption mechanisms to ensure end-to-end data confidentiality.

Acknowledgment

During the collaboration on this project, all team members engaged in active communication, thoughtful discussion, and completed their assigned tasks responsibly. Therefore, we agree that each member equally contributed to the project.

References

[1] File Transfer Protocol, Available: https://en.wikipedia.org/wiki/File_Transfer_Protocol. Accessed: Nov. 11, 2025.

[2] V. Kumar, T. Dalal, "A Review on FTP Client/Server Technology," Journal of Advance Research in Computer Science & Engineering, vol. 2, no. 6, pp. 06–10, 2024, doi: 10.53555/nncse.v2i6.443.

[3] M. Tmeizeh, C. Rodríguez-Domínguez, and M. V. Hurtado-Torres, "File chunking towards on-chain storage: a blockchain-based data preservation framework," Cluster Computing, vol. 27, no. 10, pp. 13531–13546, Dec. 2024, doi: 10.1007/s10586-024-04646-6.

[4] A. Sharma, V. Shrivastava, A. Pandey and A. Sharma, "Providing authentication using json web tokens for enhancing user security," International Journal of Research Publication and Reviews, vol. 5, no. 4, pp. 5309-5312, 2024.

[5] P. Prajapati and P. Shah, "A Review on Secure Data Deduplication: Cloud Storage Security Issue," Journal of King Saud University - Computer and Information Sciences, vol. 34, no. 7, pp. 3996-4007, 2022, doi: 10.1016/j.jksuci.2020.10.021.